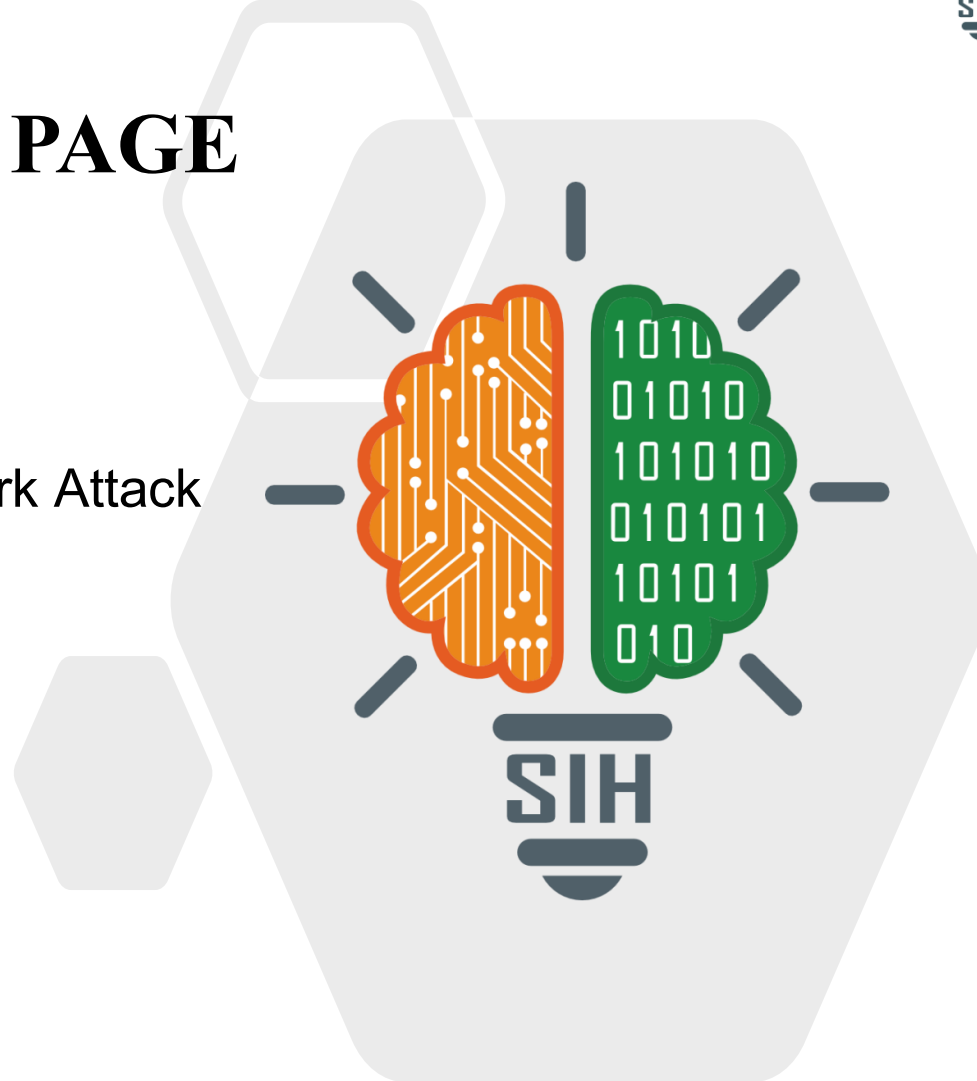


TITLE PAGE

- **Problem Statement ID** – SIH26153
- **Problem Statement Title**- AI based Network Attack
Forecasting from Network Traffic Data
- **Theme**- Blockchain & Cybersecurity
- **PS Category**- Software
- **Team ID**- 127364
- **Team Name (Registered on portal)**- VIGHNAX





VIGHNAX: a world model that forecasts network attacks before they complete



❖ Proposed Solution (Describe your Idea/Solution/Prototype)

❖ Detailed explanation of the proposed solution

- A world model (recurrent state-space model) learns how a network's state evolves: every 10 s it reads a 78-feature state built from flow records and/or packet captures
- It simulates 32 possible futures 60 s ahead: the probability that an attack is about to progress, with an uncertainty band, and the MITRE ATT&CK stage it is heading to
- Every alert is explained (Integrated Gradients) and traced to the responsible hosts and flows; working prototype: offline dashboard, PCAP / NetFlow-style CSV in, forecasts out, window by window

❖ How it addresses the problem

- Moves from detection after the fact to early warning: 54 of 201 attack onsets in unseen test traffic were forecast before they began (logistic-regression baseline: 17)
- One model trained on all seven datasets named in the PS and tested only on days, captures and botnet families it never saw

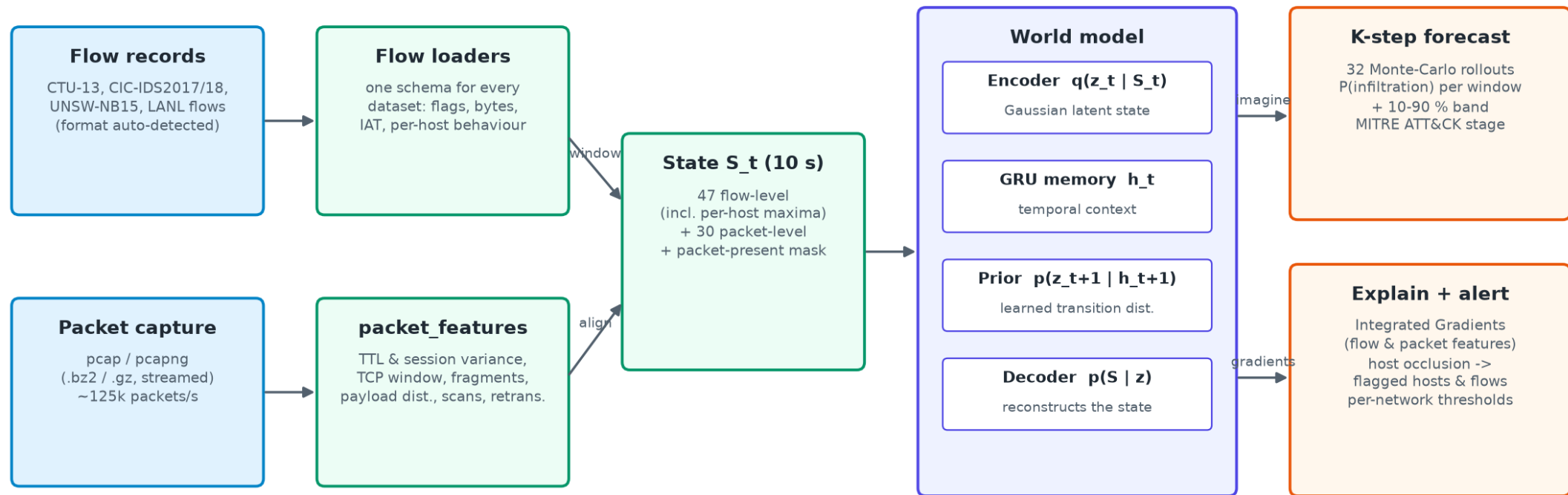
❖ Innovation and uniqueness of the solution

- Generative 'what-if' simulation of the network instead of a per-flow classifier
- Answers who, not only what: removing each host and re-running the forecast ranks the hosts behind an alert
- Air-gap ready: runs fully offline on a laptop CPU, SHA-256-verified model weights, one-click incident report

❖ Technologies to be used (e.g. programming languages, frameworks, hardware)

- **Model:** Python, PyTorch (world model), scikit-learn (baseline); dpkt / Scapy for PCAP, pandas / NumPy
- **Interface:** Flask + Chart.js dashboard with bundled fonts and scripts, no internet or cloud API
- **Hardware:** any laptop CPU (~8 ms per 10 s window); a GPU only speeds up training

❖ Methodology and process for implementation (Flow Charts/Images/ working prototype)



Offline dashboard: live timeline, forecast band, stage mix, attributions, flagged hosts & flows, incident report | no cloud APIs

❖ Analysis of the feasibility of the idea

- Working prototype, trained weights and full benchmark are already built and public on GitHub
- Unseen test data, macro over six temporal datasets: forecast AUROC 0.70 vs 0.67 (LR), detection 0.74 vs 0.72
- Throughput: one laptop CPU core processes traffic over 1,000x faster than real time

❖ Potential challenges and risks

- Weak on LANL (authentication-based red team) and behind LR on DARPA 1999; reported openly
- More false alarms on networks the model was not calibrated on; concept drift over time
- Encrypted traffic hides payloads

❖ Strategies for overcoming these challenges

- Per-network alert calibration on the site's own traffic (site profiles already built in)
- Authentication-log and per-host modelling for LANL-type attacks; periodic retraining
- Flow-level features need no payload, so the model works on encrypted traffic and on flow-only input

Unseen test data	Forecast	Detection
CTU-13	0.83 / 0.60	0.89 / 0.74
CIC-IDS2017	0.56 / 0.56	0.65 / 0.65
CSE-CIC-IDS2018	0.83 / 0.60	0.83 / 0.62
UNSW-NB15	0.95 / 0.99	1.00 / 0.95
LANL cyber1	0.37 / 0.56	0.40 / 0.55
DARPA 1999	0.63 / 0.69	0.69 / 0.81
CICIoT2023	n/a	0.99 / 0.76
Macro average	0.70 / 0.67	0.74 / 0.72

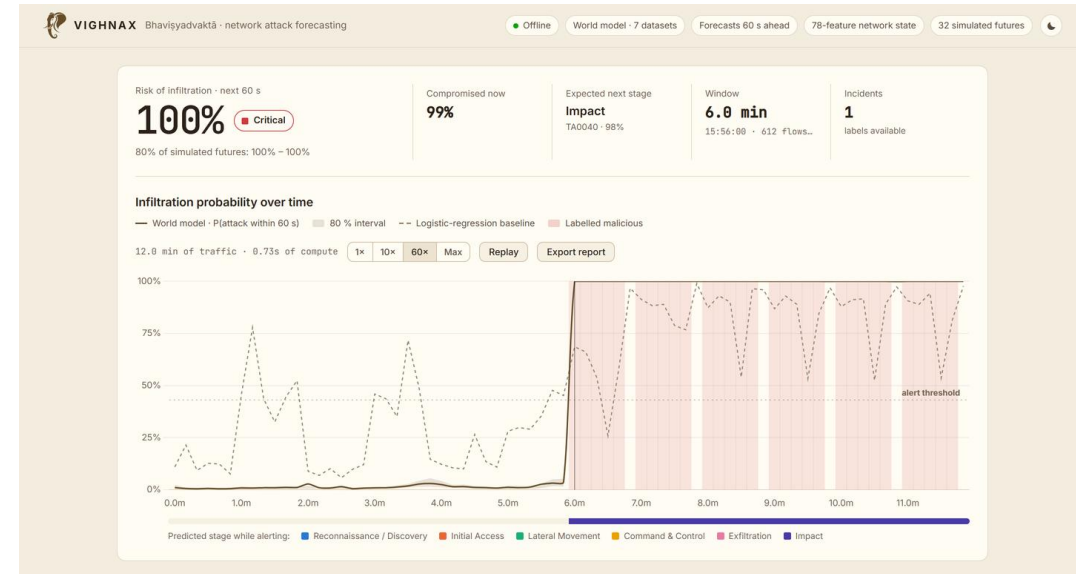
AUROC, world model / logistic-regression baseline on the same features.
CICIoT2023 episodes are composed, so only detection is scored.

❖ Potential impact on the target audience

- NTRO, NCIIPC, CERT-In and critical-infrastructure SOCs get a warning before an attack completes, with time to isolate hosts
- Every alert arrives with its reasons and the responsible hosts and flows, so triage is faster
- Runs air-gapped on ordinary hardware, including classified and low-resource networks

❖ Benefits of the solution (social, economic, environmental, etc.)

- **Security:** earlier containment of botnets, DDoS and lateral movement on national infrastructure
- **Social:** protects power, banking, health and e-governance services citizens rely on
- **Economic:** open source, no licence or cloud cost; one CPU core covers a network segment
- **Sovereignty:** indigenous, auditable AI; verified weights; no data leaves the network



Working prototype: a real CIC-IDS2017 day never seen in training. Risk stays low for six minutes, then goes critical as the DDoS starts; the baseline (dashed) raises false spikes.

❖ Details / Links of the reference and research work

- **Source code, weights, benchmark:** <https://github.com/NAMAN-THAKUR-1944/SIH2026-SIH26153-VIGHNAX>
- **World models:** Ha & Schmidhuber, World Models (2018); Hafner et al., Learning Latent Dynamics for Planning from Pixels, ICML 2019 (recurrent state-space model)
- **Explainability:** Sundararajan, Taly & Yan, Axiomatic Attribution for Deep Networks, ICML 2017 (Integrated Gradients)
- **Attack stages:** MITRE ATT&CK Enterprise matrix, attack.mitre.org
- **Datasets:** CTU-13 (Garcia et al., 2014); CIC-IDS2017 (Sharafaldin et al., 2018); CSE-CIC-IDS2018 (CIC & AWS, 2018); UNSW-NB15 (Moustafa & Slay, 2015)
- LANL cyber1 (Kent, 2015); DARPA 1999 (Lippmann et al., 2000); CICIOT2023 (Neto et al., 2023)
- **Guidance in the PS:** NCIIPC, nciipc.gov.in
- **Reproducibility:** one configuration (configs/default.yaml, seed 42); python evaluate.py regenerates every number in this deck from the public datasets